End-to-End Twitter Sentiment Analysis System

Final Project Report
Student: Hiren Patel
Program: DAAI (Data Analytics and Artificial Intelligence), Willis College
Date: October 12, 2026
GitHub: https://github.com/hirenpatel1404/sentiment140-mlops
Docker Hub: https://hub.docker.com/r/hirenpatel9/sentiment140-api

1. Introduction

Every day, people share their opinions about products, brands, events and services on social media. For a business these opinions are valuable feedback, but there are far too many to read by hand. A sentiment analysis system solves this by automatically reading a short piece of text and deciding whether the writer feels positive, negative or neutral. A marketing team can use it to track how customers react to a new campaign, a support team can use it to spot angry customers early, and a product team can use it to measure how a launch is received.

In this project I built a complete sentiment analysis system the way a machine learning engineer would in a real company, not just a model in a notebook. I cleaned historical Twitter data, trained a logistic regression model and validated it on data it had never seen. I turned the model into a Flask web service that any application can call, packaged it in a Docker container and published the image on Docker Hub. Then I added automated testing and deployment with GitHub Actions, documented how models are versioned and rolled back, and deployed the service on Kubernetes, which keeps it running and scales it automatically when traffic changes.

2. System architecture

The project has two halves. The machine learning half (top row) turns raw tweets into a trained, versioned model and serves it through an API in a container. The MLOps half (bottom row) makes sure every change is tested automatically, published as a versioned image, and deployed on Kubernetes in a way that can scale and be rolled back.

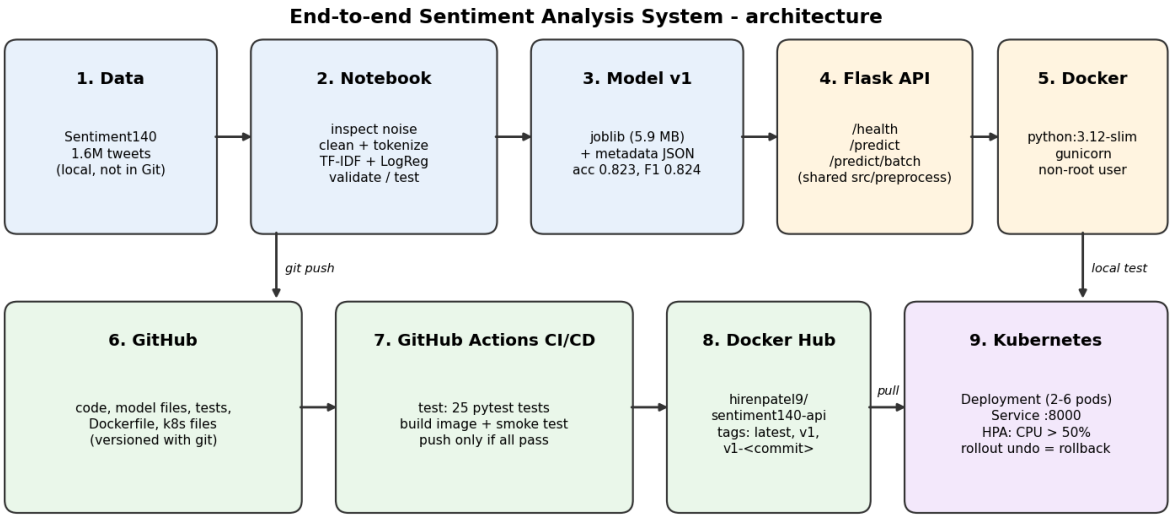


Figure 1: Architecture of the sentiment analysis system

3. Environment setup

I worked on a Windows laptop with an Intel Core Ultra 9 processor, using Git Bash as the terminal, Jupyter Notebook for analysis, and Docker Desktop for containers and Kubernetes. The main tools and versions were:

- Python 3.14.2 on the laptop and Python 3.12 inside the container, with pandas 3.0.2, NumPy 2.4.4 and scikit-learn 1.8.0
- Flask 3.1.3 for the API, gunicorn 23.0.0 as the production web server, and pytest 9.1.1 for testing
- Docker Desktop with its built-in Kubernetes (v1.36) and metrics-server for autoscaling
- GitHub for version control and GitHub Actions for CI/CD (Continuous Integration and Continuous Deployment)

The project brief also mentions TensorFlow or PyTorch for BERT. For this system I followed the main instruction to use logistic regression, and I discuss BERT as the next improvement in section 13. scikit-learn is pinned to exactly 1.8.0 everywhere, because a model saved with one scikit-learn version must be loaded with the same version.

The repository is organised so that each part of the system has its own folder: notebooks/ (analysis and training), src/ (shared text cleaning), models/ (versioned model files), app/ (Flask API), tests/ (pytest), k8s/ (Kubernetes files), .github/workflows/ (CI/CD), docs/ (versioning guide, challenges log, screenshots) and reports/ (charts). The 230 MB dataset is kept out of Git with .gitignore.

4. Data collection and preprocessing

4.1 Getting the right data

The file attached to the course page turned out to be image data with pixel columns, not tweets. I downloaded the real Sentiment140 file (training.1600000.processed.noemoticon.csv) and checked it had the expected six columns (label, id, date, query, user, text). It contains 1,600,000 tweets, exactly 800,000 negative (label 0) and 800,000 positive (label 4), with no missing values. The file uses the Latin-1 text encoding, so I loaded it with encoding='latin-1'. I recoded the labels as 0 = negative and 1 = positive. The query column only contains NO_QUERY, and id, date and user say nothing about feelings, so only the tweet text is used as input.

4.2 Inspecting the text before cleaning

Before deciding how to clean the tweets, I measured what kind of noise they contain. Punctuation appears in 77.5% of tweets, @mentions in 46.2%, apostrophes (can't, I'm) in 29.5%, digits in 22.9%, words in capitals in 14.6%, stretched letters such as "Awww" in 9.0%, HTML codes such as & in 5.9%, links in 4.7%, hashtags in 2.2% and non-English characters in 0.9%. This told me which steps matter most. Mentions and links carry no sentiment and must go, and apostrophes need careful handling because many of them are negations.

4.3 Cleaning and tokenizing

All cleaning lives in one shared file, src/preprocess.py, which both the training notebook and the Flask API import. This avoids training/serving skew, where the model sees differently cleaned text in production than in training. The steps are:

- decode HTML entities and convert to lowercase
- remove URLs and @mentions, and keep hashtag words without the # sign
- remove apostrophes first, so "can't" becomes "cant" instead of "can t"
- shorten letters repeated three or more times ("sooooo" becomes "soo")
- remove numbers, emojis, punctuation and other special characters
- tokenize into words and drop one-letter tokens except "i" and "a"

Cleaning all 1.6 million tweets took about 9 seconds. Running the same noise checks on the cleaned text found 0 remaining cases of every type, which proves the cleaning worked. I then removed 3,806 tweets that became empty, plus duplicates and 2,225 texts that appeared with both labels: 89,489 rows (5.6%) in total. That left 1,510,511 tweets, still balanced at 50.3% negative and 49.7% positive.

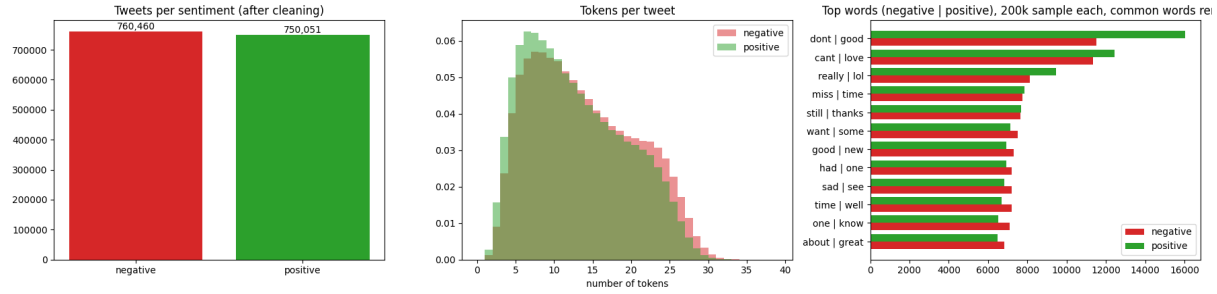


Figure 2: Class balance, tweet length and most frequent words after cleaning

Figure 2 shows that the cleaned data is balanced, so no class weights are needed. Positive tweets are slightly shorter (median 11 vs 12 tokens), and negative tweets have a longer tail, because people tend to write more when they complain. Negative tweets are dominated by "dont", "cant", "miss" and "sad", and positive tweets by "good", "love", "lol" and "thanks". "good" also appears in the negative list ("not good"), which shows that single words are not enough and motivated the use of word pairs.

5. Model training and validation

I split the data into 80% training (1,208,408 tweets), 10% validation (151,051) and 10% test (151,052), stratified so every part keeps the same balance. Text was turned into numbers with TF-IDF (Term Frequency-Inverse Document Frequency) using single words and word pairs, keeping terms that appear in at least 5 tweets, up to 300,000 features. A logistic regression model then learns one weight per feature.

I tuned the regularisation strength C on the validation set. Validation accuracy rose from 0.8150 (C = 0.25) to 0.8219 (C = 1.0) and stayed flat at C = 2.0, while training accuracy kept climbing from 0.830 to 0.865. In other words, a larger C only memorised the training data more. C = 1.0 was chosen.

On the validation set the final model reached 82.19% accuracy, an F1-score of 0.8224, precision 0.8145, recall 0.8306 and a ROC AUC (Area Under the ROC Curve) of 0.901. On the untouched test set it reached 82.31% accuracy, F1 0.8236 and AUC 0.902, almost identical to validation, so the model generalises well to new tweets. Both classes are handled equally well (F1 0.823 negative and 0.824 positive).

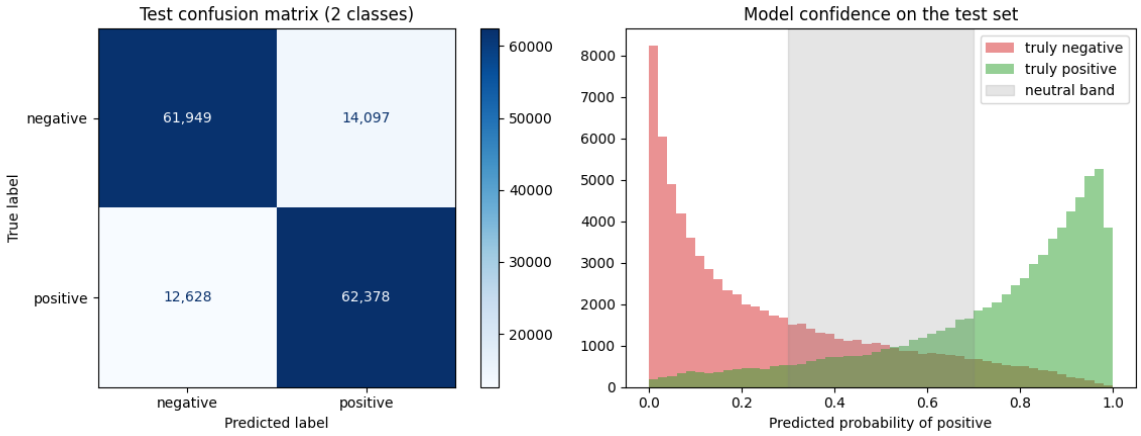


Figure 3: Test confusion matrix and the model's confidence on the test set

The confusion matrix shows 61,949 negative and 62,378 positive tweets classified correctly, with 14,097 false positives and 12,628 false negatives, so the two kinds of error are similar in size. The histogram shows that most tweets get a confident score near 0 or 1, and that errors concentrate in the middle, where the two classes overlap.

5.1 Adding the neutral class

Sentiment140 only has positive and negative labels, but the system must also answer "neutral". I use the model's confidence. A tweet with no words at all receives a positive probability of 0.672, which is the model's intercept (its starting point). So a sensible neutral band must contain 0.672, and the smallest symmetric band around 0.5 that does is 0.30 to 0.70. Tweets inside this band are labelled neutral. On the test set this labels 26.8% of tweets as neutral, and the remaining positive and negative answers are correct 89.7% of the time, up from 82.3%.

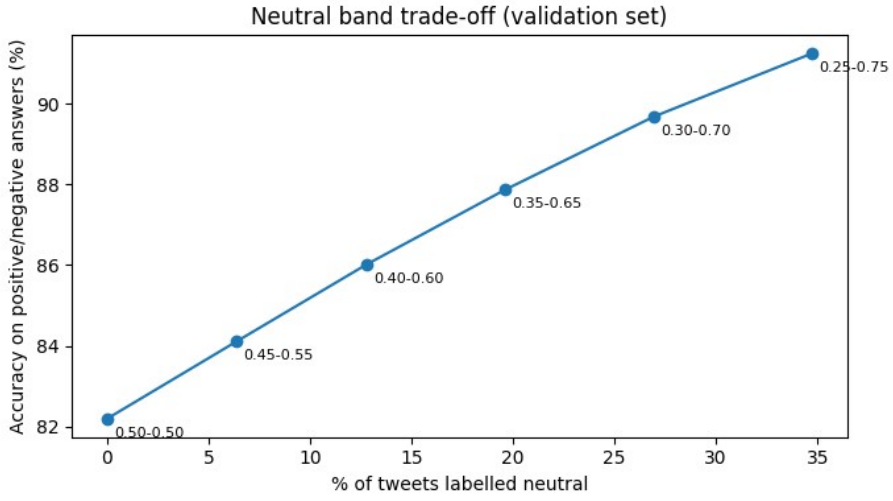


Figure 4: Trade-off between the share of neutral answers and accuracy on the rest

In practice the system behaves sensibly. "I love this phone, the camera is amazing!" is positive (0.976), and "Worst customer service ever" is negative (0.074). "The meeting is on Tuesday", "Going to the store at 5pm", "The report is attached" and "this is ok" are all neutral. "It is raining today" and "I have an exam tomorrow" come out negative, because in 2009 people mostly tweeted about rain and exams when complaining.

5.2 What the model learned

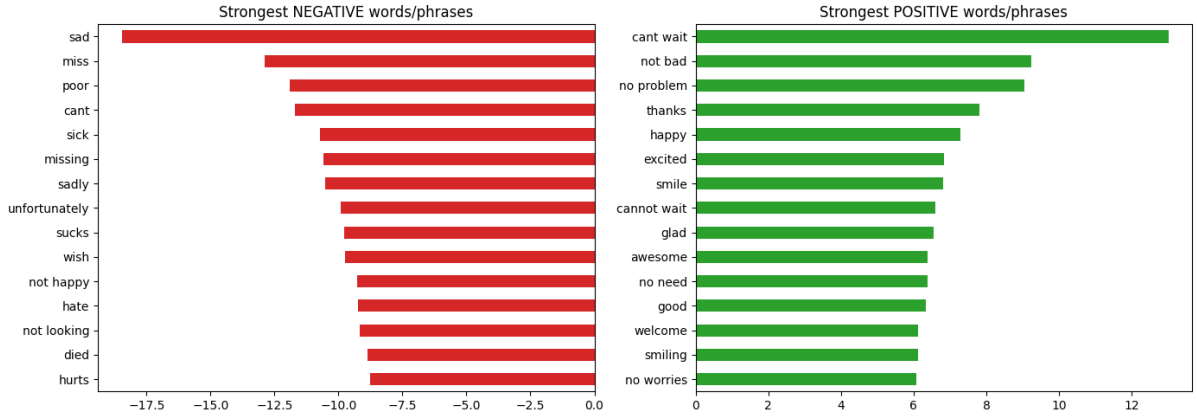


Figure 5: Strongest negative and positive words and phrases

The strongest negative signals are "sad", "miss", "poor", "cant", "sick" and "hate". The strongest positive phrase is "cant wait", followed by "not bad", "no problem", "thanks" and "happy". This confirms that word pairs and the apostrophe handling were worth it. "cant" alone is strongly negative, but "cant wait" is the most positive phrase in the model. The most confident mistakes were mostly label noise: Sentiment140 labels were created automatically from emoticons, so a tweet like "good luck love" is labelled negative.

The model was saved as models/sentiment_model_v1.joblib (5.9 MB, about 0.2 ms per prediction) together with a metadata file containing its version, settings, data fingerprint, metrics and neutral band.

6. API development

The Flask API (app/app.py) loads the model version given by the MODEL_VERSION environment variable, and reads the neutral band from the model's metadata. It cleans incoming text with the same shared function used in training. It offers four endpoints:

- GET / : a short description of the service
- GET /health : status, the loaded model version and its test accuracy
- POST /predict : one tweet in, sentiment out, with confidence, probability, cleaned text, model version and latency
- POST /predict/batch : up to 100 tweets in one request

Input is validated. A missing or non-string text field, an empty text, text longer than 1,000 characters, or a batch that is empty or larger than 100 returns HTTP 400 with a clear error message. Valid requests return HTTP 200. Example responses from the running service:

```
GET /health  ->  {"model_version":"v1","status":"ok","test_accuracy":0.8231}
POST /predict {"text":"The meeting is on Tuesday"}
  ->
{"sentiment":"neutral","prob_positive":0.6715,"confidence":0.657,"clean_text":"the
meeting is on tuesday", ...}
POST /predict {"text":""}  ->  400 {"error":"'text' must not be empty"}
```

7. Testing

I wrote 25 automated tests with pytest. Six unit tests check the cleaning function (URLs, mentions and HTML removed, lowercase, negations kept, hashtag words kept, stretched words shortened, empty input). Nineteen API tests check that the service answers /health, returns the correct sentiment for three clearly positive, three clearly negative and two neutral tweets, returns every required field, rejects seven kinds of bad input with HTTP 400, and handles batches and their limits. All 25 tests passed on my laptop in 1.6 seconds, and they pass again on every push in GitHub Actions.

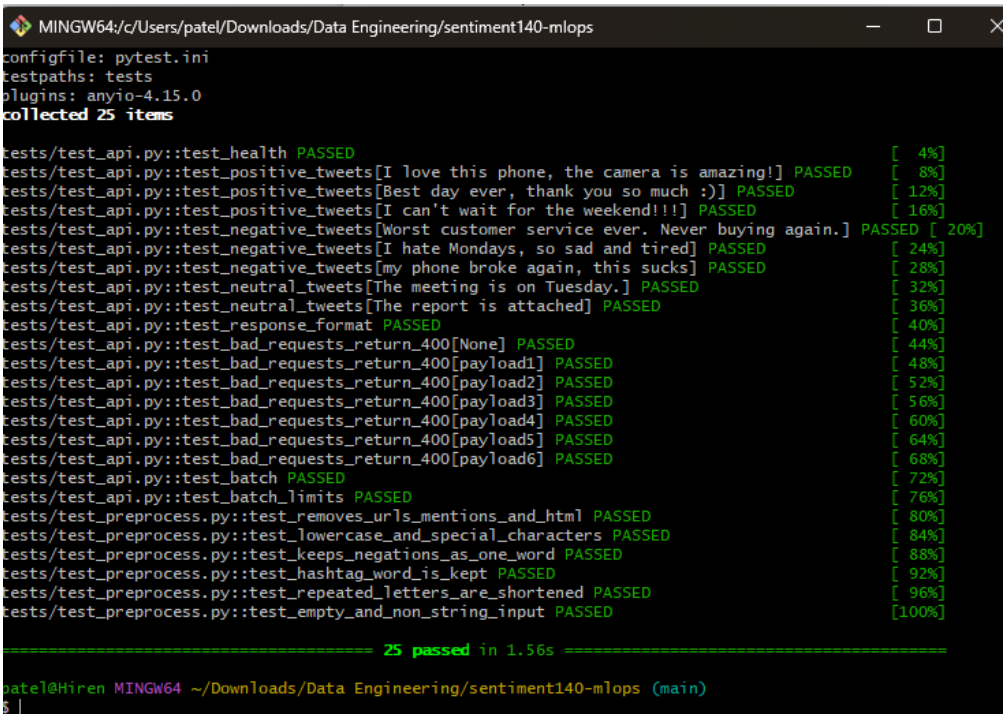


Figure 6: All 25 tests passing

8. Containerization and Docker Hub

The Dockerfile builds on the small python:3.12-slim image. It installs the pinned requirements first, so Docker can reuse that layer when only the code changes. It then copies only what the API needs (src/, app/ and models/), runs as a non-root user for safety, defines a health check against /health, and starts the API with gunicorn and 2 workers. A .dockerignore keeps data, notebooks, tests and git history out of the image. The container answered correctly on my laptop, for example classifying "I hate waiting in long lines" as negative with confidence 0.93.

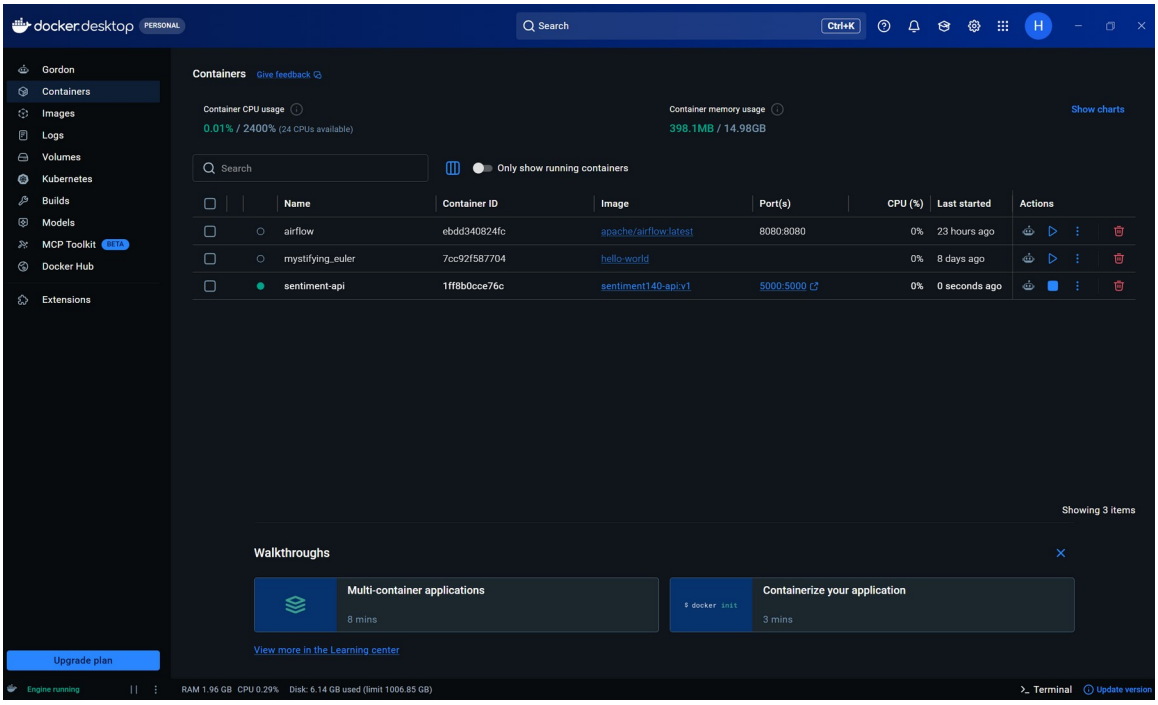


Figure 7: The container running in Docker Desktop

I pushed the image to Docker Hub as hirenpatel9/sentiment140-api (about 124 MB). It is public, so anyone can run the service with one command: docker run -p 5000:5000 hirenpatel9/sentiment140-api:v1.

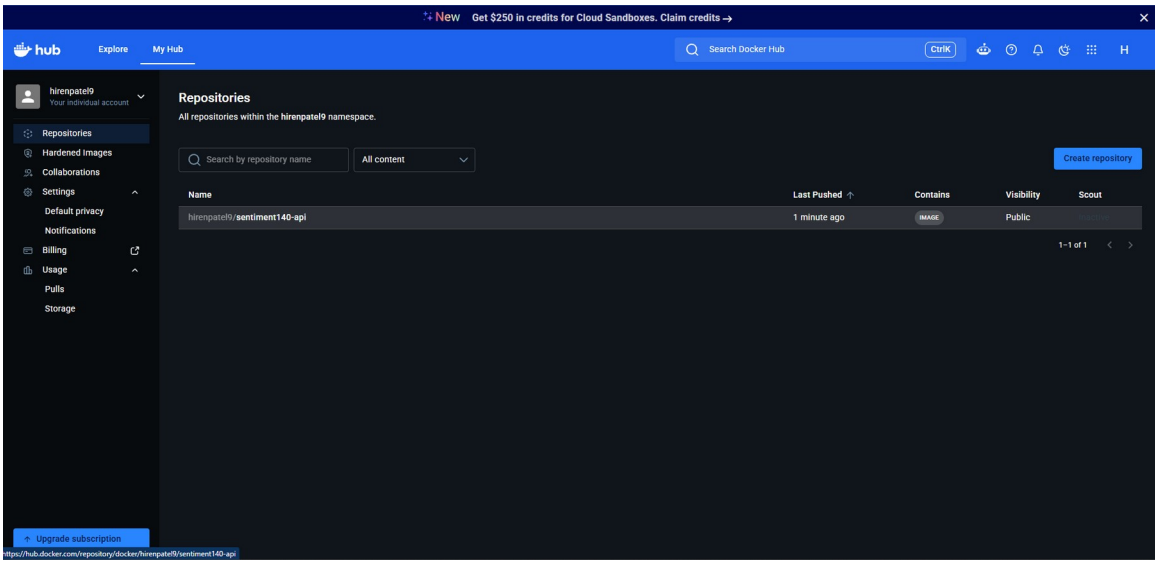


Figure 8: The image published on Docker Hub

9. CI/CD with GitHub Actions

The workflow .github/workflows/ci-cd.yml runs on every push and pull request to main. The first job (CI) installs Python 3.12 and the dependencies and runs all 25 tests. The second job (CD) runs only on a push and only if every test passed. It builds the Docker image, starts it and sends a real tweet as a smoke test ("I love this so much" must come back positive), then logs in to Docker Hub with secrets stored in GitHub and pushes three tags: latest, the model version (v1), and the model version plus the commit ID (for example v1-220149e). The first run succeeded in 1 minute 19 seconds (tests 27 s, build and push 47 s). Because tests and the smoke test run first, a broken build can never reach Docker Hub.

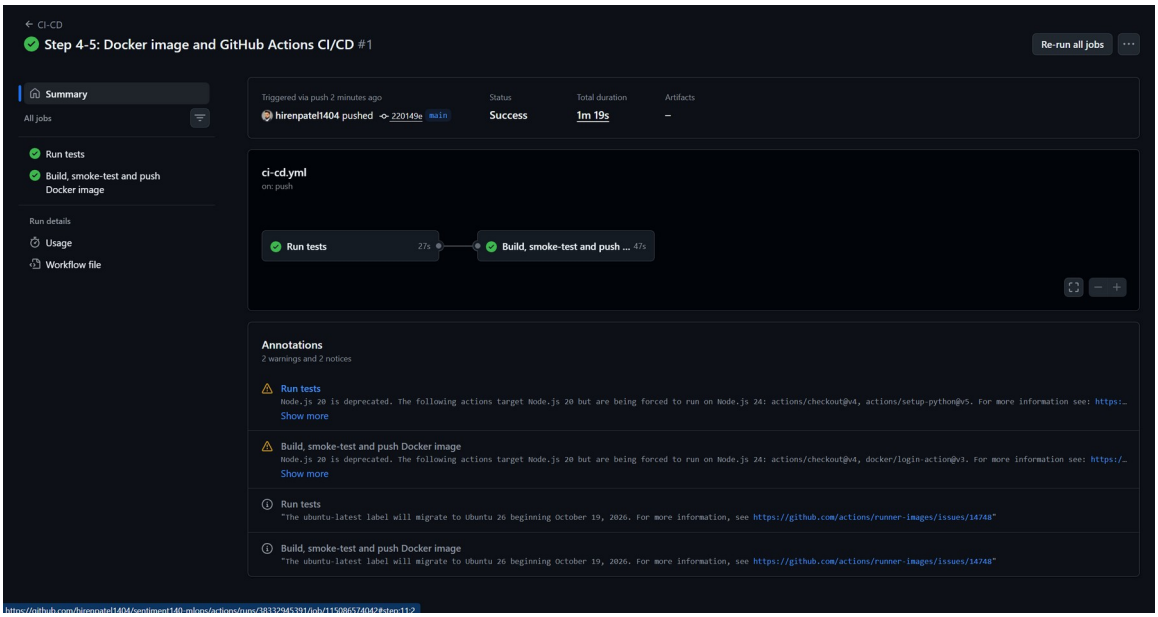


Figure 9: Successful CI/CD run on GitHub Actions

10. Model versioning and rollback

Every layer of the system is versioned. The code is versioned with git commits. Each model is saved as its own file (sentiment_model_vN.joblib) with a metadata file recording its settings, metrics and the SHA-256 fingerprint of the training data. Docker images carry three tags, and the vN-<commit> tag never moves, so any past build can be run again exactly. Kubernetes keeps the last 10 rollouts. The /health endpoint reports which model is live, so I can always check what is serving traffic.

A new model is released by training it as v2 (the v1 files stay untouched), comparing its test F1 with v1, pointing the Dockerfile and deployment at v2, and pushing. CI/CD then tests, builds and publishes it, and Kubernetes performs a rolling update with no downtime. The guide docs/MODEL_VERSIONING.md describes four rollback options: undo the last Kubernetes rollout, pin an exact image tag, switch only the model with the MODEL_VERSION environment variable, or make the rollback permanent with git revert.

I tested a rollback for real. I updated the deployment to the CI-built image v1-220149e, which Kubernetes rolled out one pod at a time, and then ran kubectl rollout undo. The deployment returned to hirenpatel9/sentiment140-api:v1 within seconds, and /health kept answering throughout.

11. Deployment on Kubernetes

I enabled Kubernetes in Docker Desktop and deployed the image with three files in k8s/. The Deployment runs 2 pods with CPU and memory requests and limits, readiness and liveness checks on /health, and a rolling update strategy that never takes a pod down before its replacement is ready. The Service exposes all pods behind one address, http://localhost:8000, which answered both /health and /predict correctly. The HorizontalPodAutoscaler keeps between 2 and 6 pods and adds pods when average CPU goes above 50%. I installed metrics-server so Kubernetes can measure CPU.

To test autoscaling, I started a load generator that continuously sends batches of 100 tweets. CPU jumped from 1% to 424% of the target, and Kubernetes scaled the deployment from 2 to 6 pods within about a minute. It stayed at the maximum of 6 while the load continued (CPU 132-291%). When I stopped the load generator, CPU fell to 1%, and after the 1-minute stabilisation window the deployment scaled back down to 2 pods. This shows the system adds capacity automatically under heavy traffic and releases it when traffic drops.

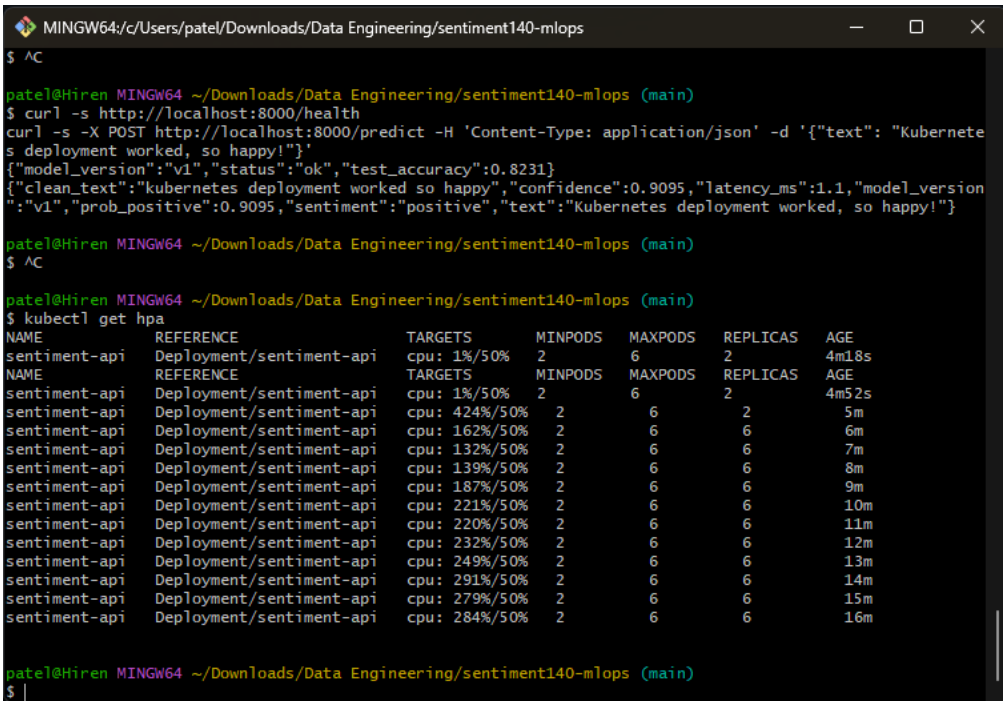


Figure 10: Autoscaler scaling from 2 to 6 pods under load

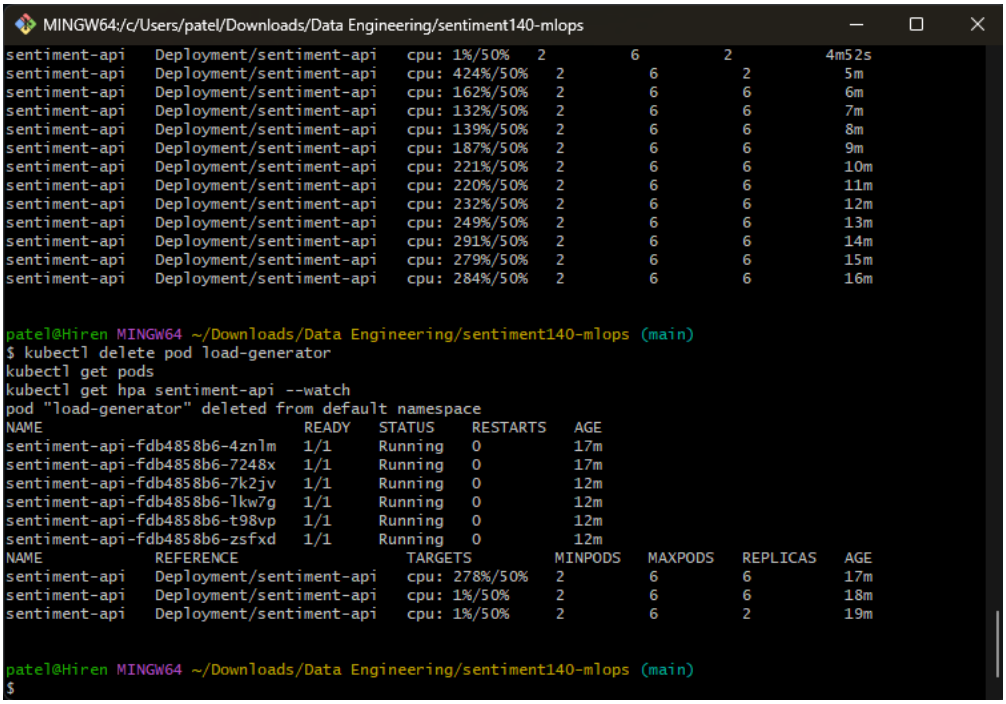


Figure 11: Six pods running under load, then scaling back to 2

12. Challenges and how I solved them

I recorded every problem in docs/CHALLENGES.md (19 entries). The most important ones were:

- The dataset on the course page was image data, so I downloaded the real Sentiment140 file and verified its structure.
- pandas 3 stores text in a new Arrow format that crashed scikit-learn's train_test_split. Converting the column to a NumPy array fixed it.
- Plain cleaning broke negations ("can't" became "can t"), so apostrophes are removed first.
- The dataset has no neutral labels, and a first 0.40-0.60 band still called factual sentences positive. Basing the band on the model's empty-tweet probability fixed it.
- The TF-IDF vectorizer stores a huge list of removed words, which I delete before saving to keep the model at 5.9 MB.
- gunicorn doesn't run on Windows, so I tested locally with Flask and pytest, and use gunicorn inside the Linux container.
- In Git Bash, "!!" inside double quotes repeats the previous command, so JSON for curl must use single quotes.
- In the rollback demo, a change-cause note added before the update was copied to both revisions. The lesson was to annotate after the change, and to update deployment.yaml after an emergency rollback so the next kubectl apply doesn't undo it.

13. Limitations and future work

The model's accuracy is limited by Sentiment140's automatic, emoticon-based labels, which contain noise, and by the 2009 language of the tweets. The neutral class is an approximation based on confidence, not on human-labelled neutral tweets. Kubernetes runs on a single local node, not in the cloud. Next steps would be to fine-tune a transformer model such as BERT and compare it with logistic regression as model v2, to label a set of neutral tweets to measure and improve the neutral class, to deploy to a managed cloud cluster (for example Amazon EKS), to update the deployment automatically from CI/CD, and to add monitoring for data drift, using the shift-detection methods from Assignment 7.

14. Conclusion

This project delivers a working, end-to-end sentiment analysis system rather than only a model. Evidence-based cleaning and a TF-IDF plus logistic regression model reached 82.3% accuracy and an F1-score of 0.824 on 151,052 unseen tweets, with a neutral class that makes the answers more useful. The model is served by a validated Flask API, packaged in a Docker image published on Docker Hub, and protected by 25 automated tests and a CI/CD pipeline that only publishes images that pass. Every model and image is versioned and can be rolled back in seconds, and the service runs on Kubernetes, where it scaled automatically from 2 to 6 pods under load and back again. Together these parts show the full workflow of a machine learning engineer, from raw data to a reliable, scalable service.

Appendix: how to run the project

```
git clone https://github.com/hirenpatel1404/sentiment140-mlops.git
cd sentiment140-mlops
python -m pip install -r requirements-dev.txt
python -m pytest -v                          # 25 tests
python app/app.py                            # local API on :5000
docker run -d -p 5000:5000 hirenpatel9/sentiment140-api:v1   # or run the published
image
kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml -f k8s/hpa.yaml   # API
on :8000
curl -s -X POST http://localhost:8000/predict -H 'Content-Type: application/json' -d
'{"text": "I love it"}'
```